

Talos Linux: um OS minimalista e sem SSH para o Kubernetes

Emerson Silva

CNCF Araraquara + FLISOL



| *"Não existe trabalho ruim, o ruim é ter que trabalhar."*

Seu Madruga

Emerson Silva

- Engenheiro DevOps/SRE na **4Linux**
- +9 anos em ambientes DevOps críticos
- Foco em **Kubernetes**, IaC e confiabilidade
- Escritor, instrutor e palestrante ativo na comunidade



O que veremos hoje

1. O problema: OS tradicional como ponto de fricção
2. O que é o Talos Linux
3. Arquitetura e filosofia
4. Segurança e atualizações
5. Armazenamento no Talos
6. Prática
7. Quando faz sentido usar

O problema com o OS tradicional

- Gerenciamento manual de pacotes e atualizações
- SSH aberto = superfície de ataque enorme
- Scripts de cloud-init frágeis
- Configuração manual de hardening
- **"Servidor que ninguém sabe como foi configurado"**

O que é o Talos Linux?

- OS **atômico e modular** construído exclusivamente para Kubernetes
- Sem SSH · Sem shell · Sem pacotes desnecessários
- Gerenciado por **API gRPC declarativa** via `talosctl`
- Sistema de arquivos **imutável** (SquashFS read-only em memória)
- Imagem com menos de **80MB**
- `machined` como **PID 1** — escrito em Go do zero

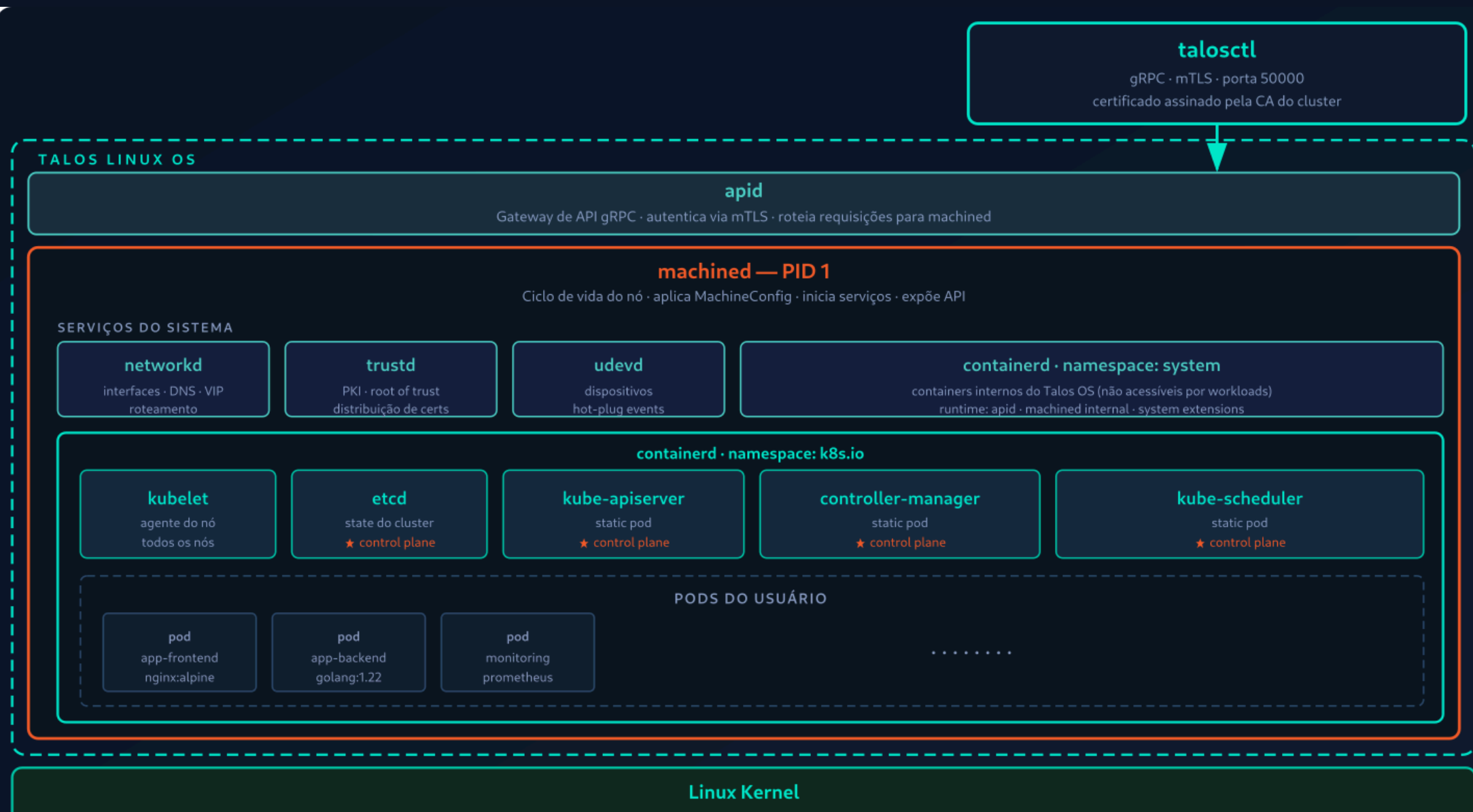


A filosofia do Talos

- **Distribuído** — HA por design, sem SPOF
- **Imutável** — sempre roda de SquashFS, zero configuration drift
- **Minimal** — sem shell, sem GNU utils, sem busybox
- **Efêmero** — tudo é reconstruível
- **Seguro** — sem senhas, mTLS, certificados auto-rotativos
- **Declarativo** — um único YAML para tudo
- **Cattle, not pets** — nós substituídos, nunca reparados

Arquitetura do Talos Linux

8 / 22



Sem SSH. Sem shell. Sério.

```
# Tentativa de acesso via SSH  
ssh root@192.168.121.72  
# ssh: connect to host 192.168.121.72 port 22: Connection refused  
  
# A forma correta: API via talosctl  
talosctl -n 192.168.121.72 get members  
talosctl -n 192.168.121.72 dashboard  
talosctl -n 192.168.121.72 logs machined
```

“ Toda interação acontece via API gRPC autenticada por certificado ”

Arquitetura do sistema de arquivos

Partição	Ponto de montagem	Conteúdo
EFI	—	Dados de boot EFI
BOOT	—	Bootloader, initramfs, kernel
STATE	<code>/system/state</code>	MachineConfig, certificados
EPHEMERAL	<code>/var</code>	Dados do K8s, containerd, etcd, logs

- Raiz: **SquashFS read-only** montado em memória
- Sem instalação de pacotes em runtime
- Reset = disco limpo e cluster de volta em minutos
- `/var` sobrevive a reboots, mas é apagado em `reset` sem `--preserve`

Init system: machined vs systemd

	Linux Tradicional	Talos Linux
PID 1	<code>systemd</code>	<code>machined</code>
Função	Gerencia serviços	Ciclo de vida completo do nó
Serviços	Units <code>.service</code>	<code>containerd</code> + <code>kubelet</code> via API
Configuração	<code>/etc/systemd/</code>	MachineConfig YAML
Acesso	<code>systemctl</code> + SSH	<code>talosctl</code> via gRPC

“ `machined` monta o FS, aplica a config, inicia os serviços e expõe a API — tudo sem shell ”

Configuração declarativa

```
# Um único YAML define tudo
machine:
  network:
    interfaces:
      - deviceSelector:
          physical: true
          dhcp: true
          vip:
            ip: 192.168.121.100
cluster:
  network:
    cni:
      name: flannel
```

Segurança por design: mTLS

talosctl → [certificado cliente] → API Talos



mTLS bidirecional

[CA do cluster] ← autentica ambos os lados

- Toda chamada à API exige **certificado assinado pela CA do cluster**
- Permissões **granulares por certificado** (ex: read-only para monitoramento)
- Sem SSH = sem **CVE-2024-6387** (regreSSHion) e similares
- Sem **apt** / **yum** = sem instalação de malware ou backdoors em runtime
- **Superfície de ataque mínima**: shell, gerenciadores de pacotes e SSH removidos por design

Atualizações atômicas com boot A/B

```
# Atualiza o nó para uma nova versão do Talos  
talosctl upgrade --nodes 192.168.121.72 \  
  --image ghcr.io/siderolabs/installer:v1.9.0
```

- **Kernel embutido na imagem** — userspace e kernel sempre compatíveis
- Sistema de boot **A/B**: slot ativo e slot de standby
- Falha no upgrade → **rollback automático** para a versão anterior
- Atualizar o kernel = substituir a imagem completa do sistema
- Zero drift: o SO é sempre idêntico ao que foi testado

Armazenamento: opções CSI recomendadas

Solução	Caso de uso	Destaque
Longhorn	Simplicidade / KubeVirt	Redundância + LiveMigration
Rook + Ceph	Escala / larga escala	Escala "infinita", mais complexo
OpenEBS Mayastor	Alta performance	NVMe-TCP, baixa latência
AWS EBS / GCP PD	Nuvem	CSI nativo do provedor
NFS / iSCSI	Bare metal / on-prem	<code>nfs-subdir-external-provisioner</code>

Dimensionamento de armazenamento

- **Control plane**: mínimo **40GB** por nó (etcd reside na partição **EPHEMERAL**)
- Clusters com **100+ nós**: mesma base, mas monitoramento contínuo é essencial
- `talosctl reset` apaga `/var` — use `--preserve` para manter dados do etcd
- Evite rodar workloads pesadas nos **mesmos nós** que processam armazenamento

```
# Diagnóstico de armazenamento sem SSH  
talosctl -n 192.168.121.72 read /proc/mounts  
talosctl -n 192.168.121.72 dmesg | grep -i disk
```

| Demo ao vivo

O que vimos na demo

Subindo o cluster

```
talosctl cluster create docker --workers 2 --name talos-lab
```

Verificando membros via API

```
talosctl get members --nodes 10.5.0.2
```

Dashboard sem SSH

```
talosctl dashboard --nodes 10.5.0.2
```

Kubernetes funcionando normalmente

```
kubectl get nodes
```

```
kubectl get pods -A
```

Quando usar o Talos Linux?

-  Clusters Kubernetes em produção
-  Ambientes que exigem hardening e compliance
-  Infraestrutura GitOps e declarativa
-  Bare metal, VMs, cloud e homelab
-  Times que querem reduzir overhead operacional
-  Armazenamento com Longhorn, Rook/Ceph ou OpenEBS via CSI
-  Não é para workloads que precisam de SSH direto ao node
-  Curva de aprendizado inicial existe (mas vale a pena)

Comparando com o tradicional

	Ubuntu / CentOS	Talos Linux
Acesso	SSH + shell bash	API gRPC · <code>talosctl</code>
PID 1	<code>systemd</code>	<code>machined</code>
Tamanho	~1 GB+	~80 MB
Gestão	Manual / Ansible	Declarativa via YAML
Sistema de arquivos	Mutável	SquashFS read-only
Atualizações	<code>apt upgrade</code> incremental	Atômica A/B + rollback
mTLS	✗	✓ nativo
Config drift	Frequente	Impossível
Superfície de ataque	Alta	Mínima

Conclusão

01

Experimente

comece com

```
talosctl cluster  
create docker
```

02

Leia a série

emerson-
silva.blog.br

03

Seja o próximo a
eliminar SSH da
sua infra

Infraestrutura **imutável** começa com o primeiro **talosctl apply-config**

Obrigado!

Recursos

- 📖 docs.siderolabs.com
- 📝 emerson-silva.blog.br
- 💬 @silvemerson

Vamos manter contato!

